



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院

你认为软件架构是什么？

A

软件的结构

B

软件的组成方式

C

构成软件的子系统

D

软件的分层方式

E

不太清楚



提交



软件架构

- 软件系统的总体结构（包括构件及其连接关系）；
- 强调子系统/构件的外部视图；
- 是高层设计，满足功能性的同时重点考虑系统的质量属性（性能、安全性、可维护性等）；
- 是详细设计和实现的出发点。



课程介绍

● 目的

- 提高在概念化层次上进行问题理解和方案规划的能力，实现从程序员向**架构师**的转变。



Rod Johnson



尤雨溪



Gavin King



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

课程介绍

● 课程内容

- 面向**软件架构**(体系结构)设计，以**UML 语言**为基础，从基于用例的需求建模、基于类图的静态建模、基于对象交互分析的动态建模、状态机建模等基本的**需求和分析建模**手段开始，逐步介绍多种**软件架构模式**以及基于模式的**软件架构设计方法**。



先修课程或准备知识

- 面向对象编程与设计
- 数据库系统
- 网络与分布计算
- 软件工程



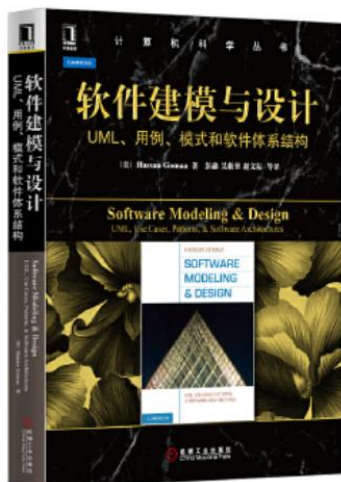
考核办法

- 考试课，成绩由以下三部分组成：
 - 平时成绩（包括课堂讨论、课堂出勤）：10%
 - 实验成绩（包括实验出勤、实验报告）：30%
 - 期末笔试：60%

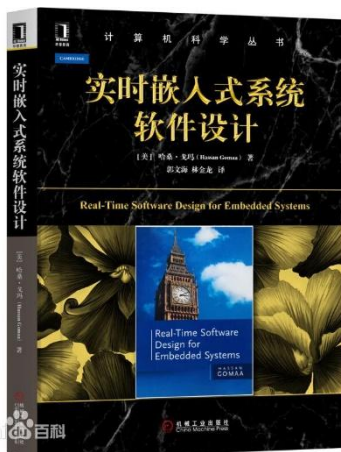


教材及资源

教材



- 《软件建模与设计：UML、用例、模式和软件体系结构》(美)Hassan Gomaa著，彭鑫、吴毅坚等译，机械工业出版社，2014.8



- 《实时嵌入式系统软件设计》(美)Hassan Gomaa著，郭文海、林金龙等译，机械工业出版社，2018.12



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

教材及资源

● 参考书

- 《软件架构：架构模式、特征及实践指南》(美)Mark R.N.F著，杨洋等译，机械工业出版社，2021
- 《软件产品线工程原理与方法》，李伟刚、李易编著，科学出版社，2015



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.



第一章 软件建模与软件架构

主要内容

- 1.1 软件模型和建模
- 1.2 软件架构
- 1.3 模型驱动的架构（MDA）
- 1.4 软件架构的视图
- 1.5 软件建模和软件架构设计的发展史
- 补充知识：
 - UML表示法



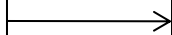
1.1 软件模型和建模



● 模型

- 来源——用于提供艺术和建筑学中的小规模规划
- 科学和工程：

在某一精度和
细节层次上提
供系统的抽象



分析模型获得
对所开发系统
的更好的理解

- **OMG**：建模就是在编码之前对软件应用的设计。在系统实现之前，对模型进行构造和分析，并用于指导后继的实现过程
- 从多个视图构造模型
- 模型驱动架构（**MDA**）



1.1 软件模型和建模

● 建模

○ 面向对象方法

封装：把客观事物封装成抽象的类，并且类可以把自己的数据和方法只让可信或必要的类或者对象操作，对不可信的或不必要的类或对象进行信息隐藏。

继承：让某个类型的对象获得另一个类型的对象的属性的方法。它可以使用现有类的所有功能，并在无需重新编写原来的类的情况下对这些功能进行扩展。

多态：类实例的相同方法在不同情形有不同表现形式。多态机制使具有不同内部结构的对象可以共享相同的外部接口。



1.1 软件模型和建模

● 建模

○ 统一建模语言（UML）

- **OMG**提出；
- 为面向对象模型的描述提供了一种标准化的图形语言和表示法；
- 与面向对象分析和设计方法一起使用；
- 几乎所有现代的面向对象方法都使用**UML**表示法来描述软件需求、分析和设计模型。

○ 核心方法

- 用例建模——需求
- 静态建模——结构
- 动态建模——行为



1.2 软件架构

- 将系统总体结构（包括构件及其连接关系）与各个构件的内部细节分离；
- 在不同的细节层次上描述；
- 强调子系统/构件的外部视图，即子系统/构件所提供和需要的接口以及与其它子系统/构件的连接关系；
- 是高层设计，满足功能性的同时重点考虑系统的质量属性（性能、安全性、可维护性等）；
- 是详细设计和实现的出发点。

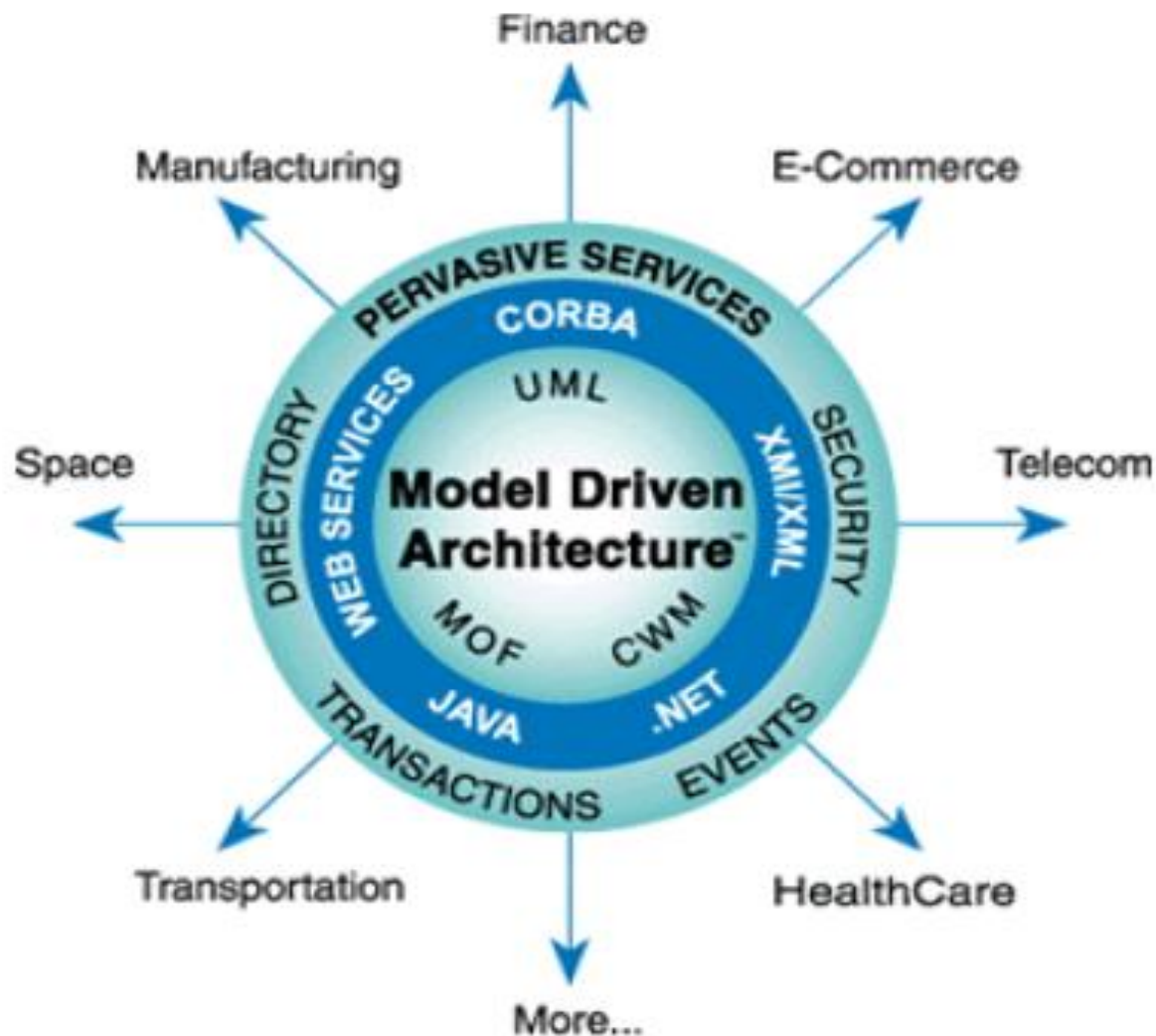


1.3 模型驱动的架构

- 为适应业务和技术的变更提供一个开放的、与厂商无关的方法
- 将对系统的操作的定义与系统使用其平台能力的详细方式分离
- 为如下方面提供了方法和使能工具
 - 为系统独立地定义支持它的平台
 - 为系统选择特定平台
 - 将系统定义转换成针对特定的平台
- 目标是
 - 通过从架构上分离关注点实现可移植, 可互操作, 可复用的目标



1.3 模型驱动的架构



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

1.3 模型驱动的架构

- **CIM: Computation Independent Model**

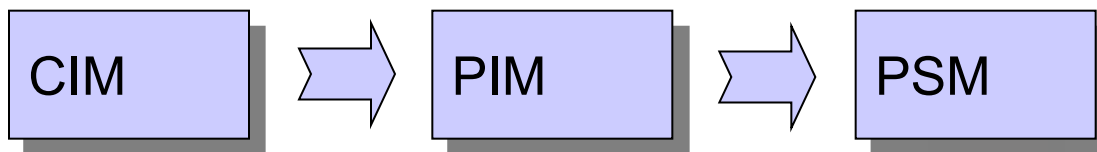
- 系统的与计算无关的视图
- 详细描述需求,但是隐藏实现细节和系统实现
- 弥补领域专家和技术专家间的鸿沟

- **PIM: Platform Independent Model**

- 系统的独立于平台的视图
- 具有充分的独立程度,以便能够映射到一个或多个平台
- 以抽象出技术细节的方式定义了一组服务

- **PSM: Platform Specific Model**

- 与平台相关的系统视图
- 将**PIM**的定义与系统如何使用特定类型的平台的详细定义相关联

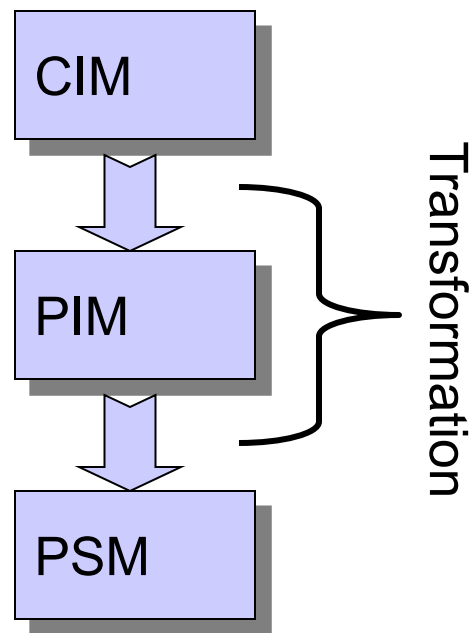


1.3 模型驱动的架构

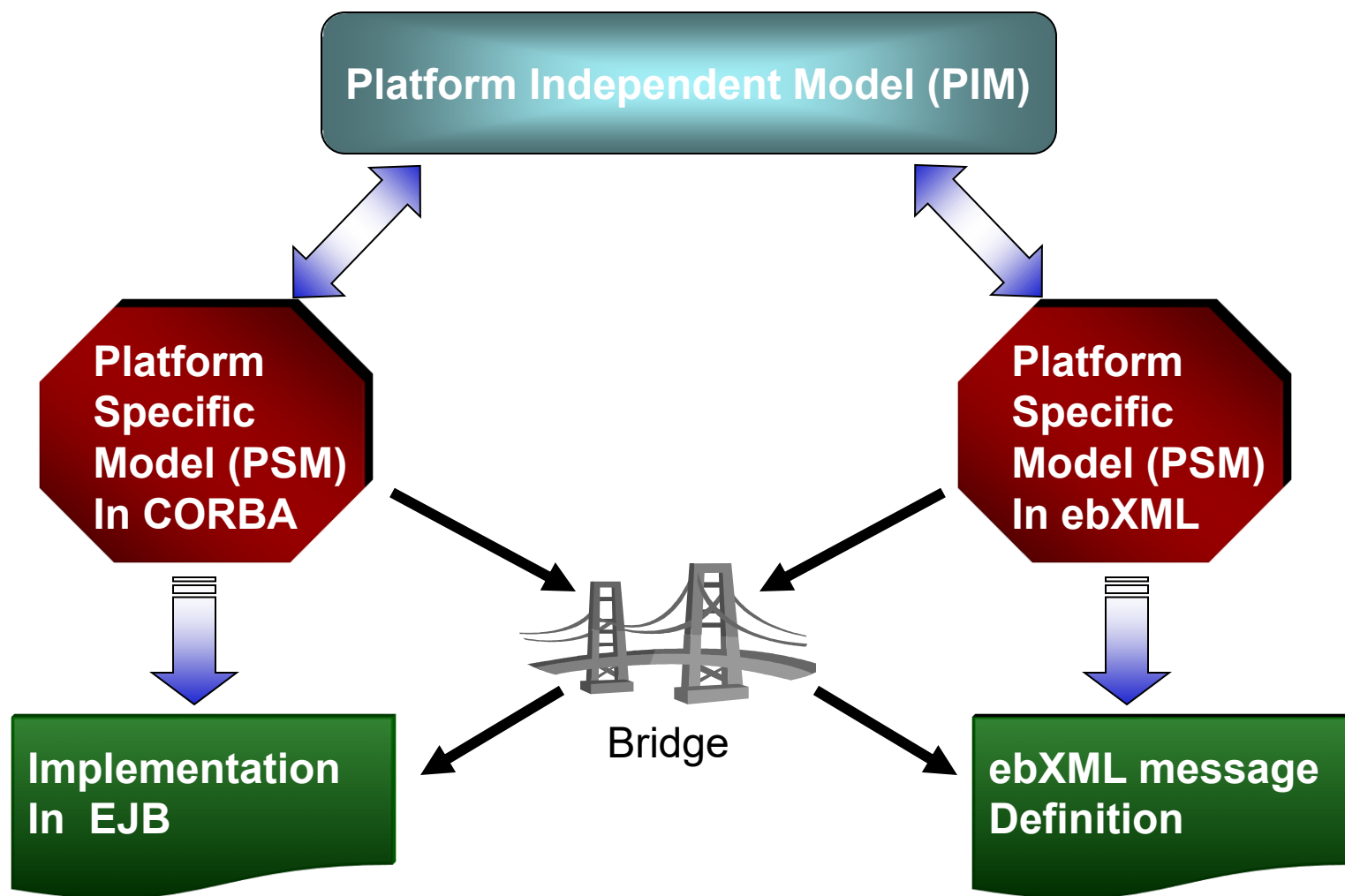
● 模型转换

○ 将同一系统的一个模型转换成另一个模型的过程

- 标识
- 元模型转换
- 模型转换
- 模式应用
- 模型合并



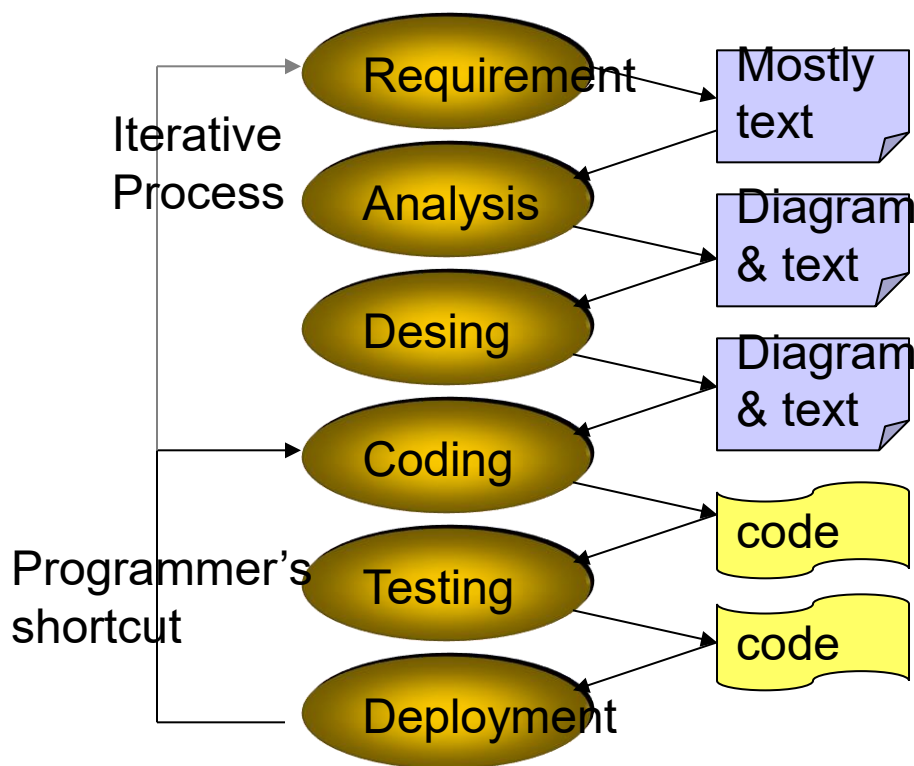
1.3 模型驱动的架构



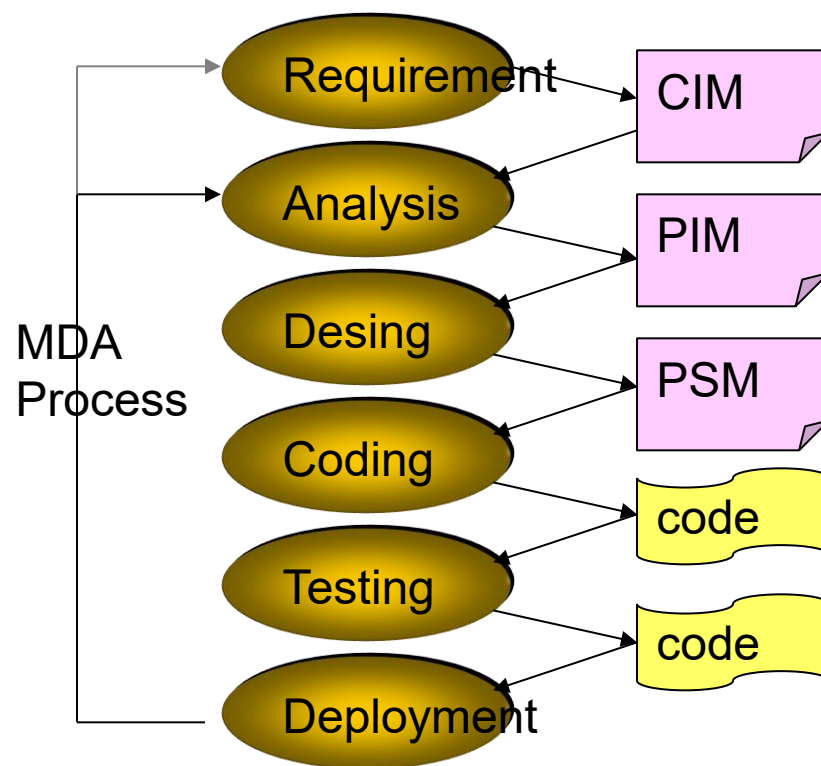
1.3 模型驱动架构

MDA对开发过程的影响

传统开发生命周期流程



MDA开发生命周期流程



1.3 模型驱动的架构

- 保护知识投资
 - 独立于实现平台
 - 约定成俗的知识被显式描述
- 加快开发速度
 - 许多实现是自动生成的
- 实现的质量
 - 专家提供了转换模版
- 维护和文档
 - 设计和分析模型不会写完就被抛弃
 - 从定义到实现**100%**的可追溯性



优点



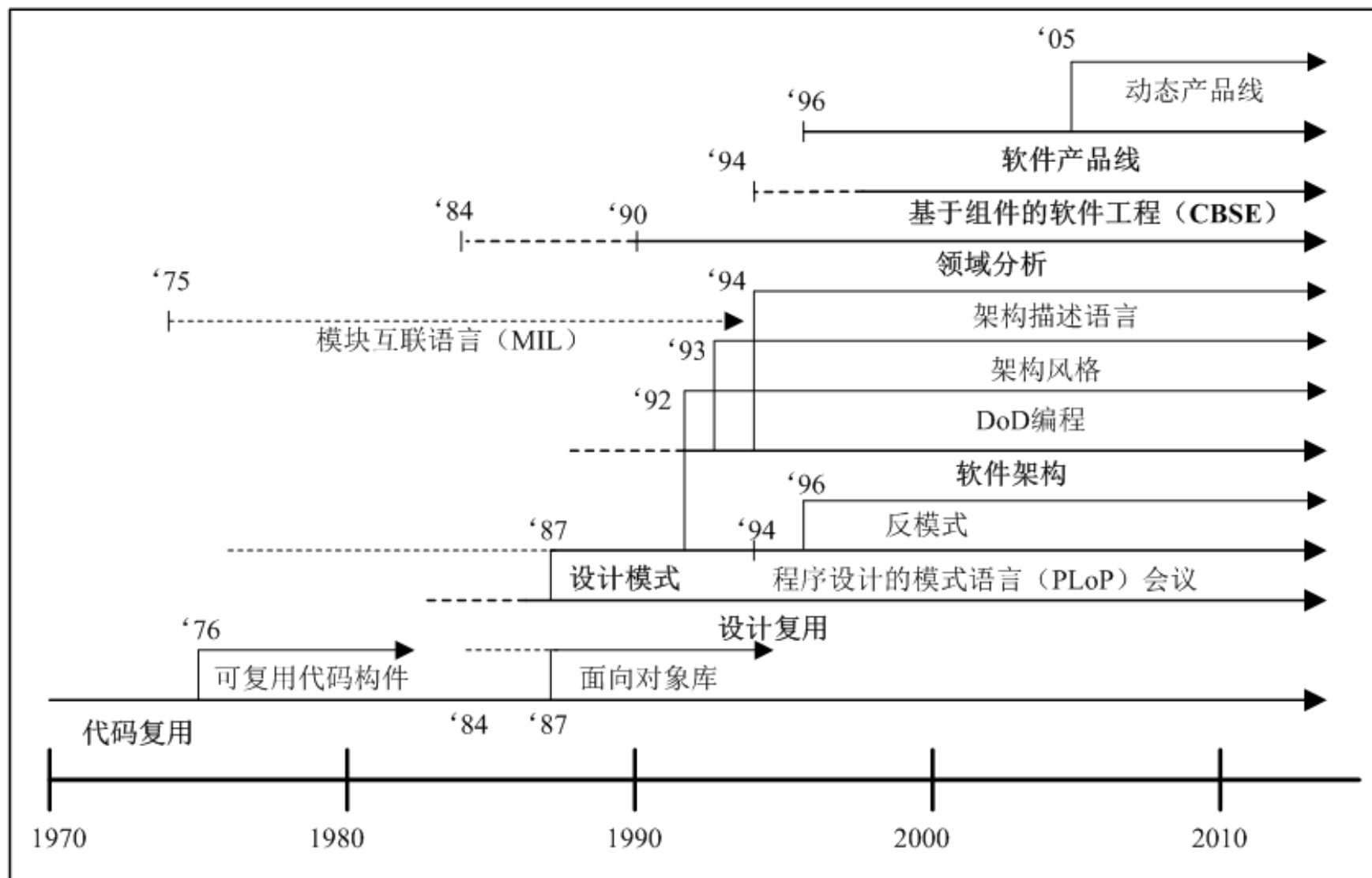
1.4 软件架构的视图

- 根据**UML**表示法，区分如下视图

- 用例视图 → 功能需求
- 静态视图 → 类及类关系
- 动态交互视图 → 对象间的消息通信
- 动态状态机视图 → 构件中的状态控制行为
- 结构构件视图 → 构件及构件互联
- 动态并发视图 → 分布式构件间消息通信
- 部署视图 → 分布式构件的部署



1.5 软件建模和软件架构设计的发展史



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

1.5 软件建模和软件架构设计的发展史

'60 ● 代码模块化：子程序或函数→流程图

'70 ● 结构化设计方法

'70中晚期

○ 面向数据流的设计方法：考察数据在系统中的流动来理解系统的功能→数据流图

○ 数据结构化设计方法：基于数据结构设计程序结构→E-R图

1972 ● 信息隐藏（**Parnas**）：减少全局数据的使用，增强系统可靠性和维护性

'80 ○ 实时结构化分析和设计方法

○ 实时系统设计方法



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.

1.5 软件建模和软件架构设计的发展史

'80早期

● **Jackson系统设计方法**：将软件设计视为对现实世界的模拟→并发任务网络

- 对现实世界的实体使用并发任务进行建模
- 突破自顶而下的设计思想→中间向外的行为性建模
- 是对象交互建模的先导

'80中晚期

● **面向对象分析和设计方法**

- 设计重点：问题域建模、信息隐藏和继承
- 提供了比结构化设计更平滑的分析到设计的过渡
- 对象建模技术**OMT→UML**

'90



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

2008-2024 by Li Weigang. All rights reserved.



补充知识

UML表示法

● 用例图

○ 用例定义

● 参与者

● 系统

● 用例

● 通信

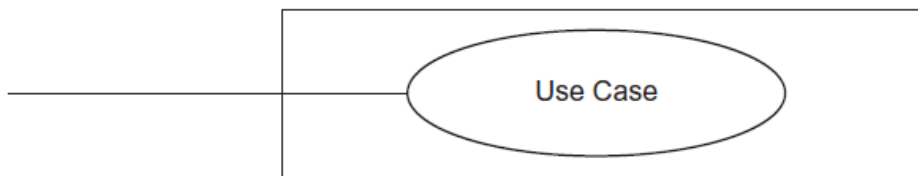
● 用例

• Ex

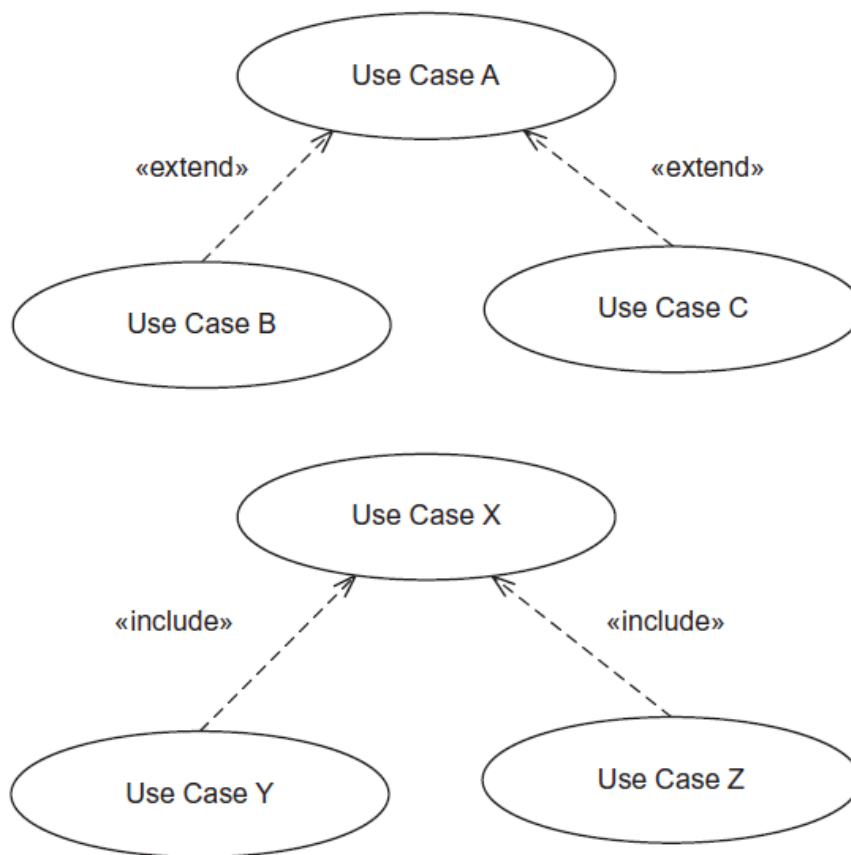
• Inc



Actor



系列



西北工业大学

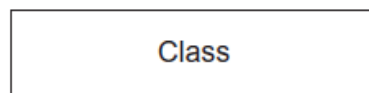
NORTHWESTERN POLYTECHNICAL UNIVERSITY

UML表示法

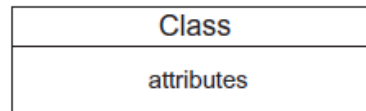
● 类/对象图

○ 类

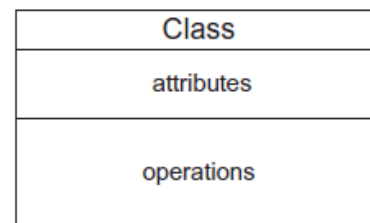
- 类名
- 属性
- 操作



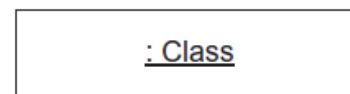
Class



Class with attributes



Class with attributes and operations



Objects

○ 对象

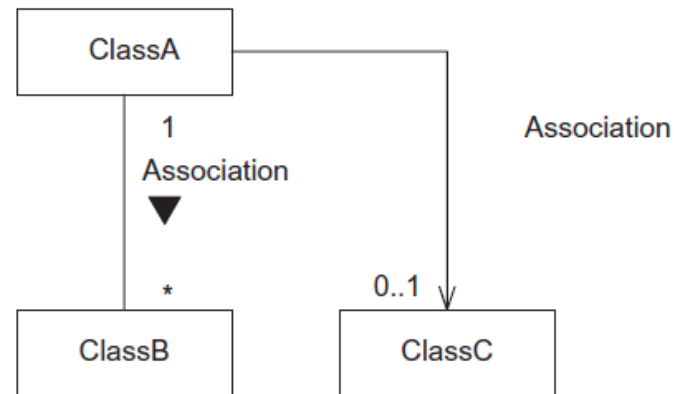
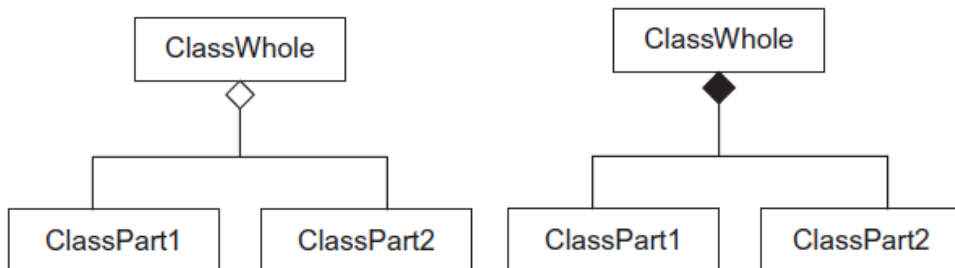
- **anObject:Class**(缩写 **anObject**或**:Class**)



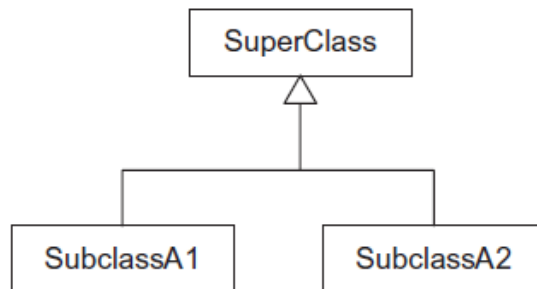
UML表示法

○ 关系

- 关联
- 整体/部分关系
- 泛化/特化关系



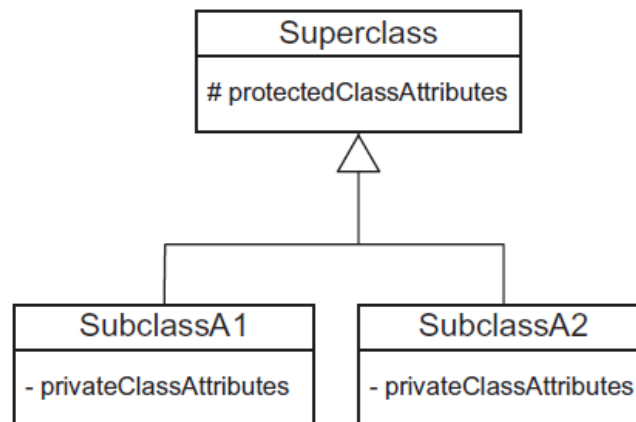
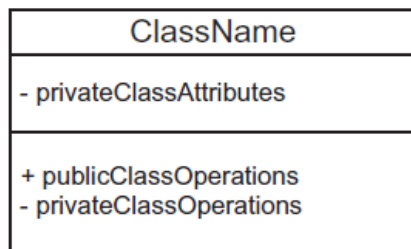
1	Class	Exactly one
0..1	Class	Optional (zero or one)
*	Class	Many (zero or more)
1..*	Class	Many (one or more)
m..n	Class	Numerically specified



UML表示法

○可见性

- 公有
- 私有
- 受保护

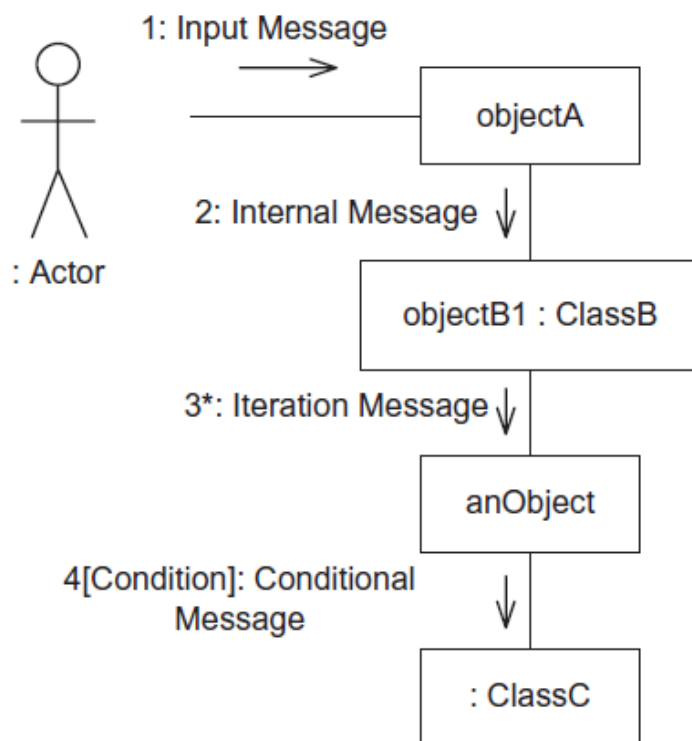


UML表示法

- 交互图

- 通信图

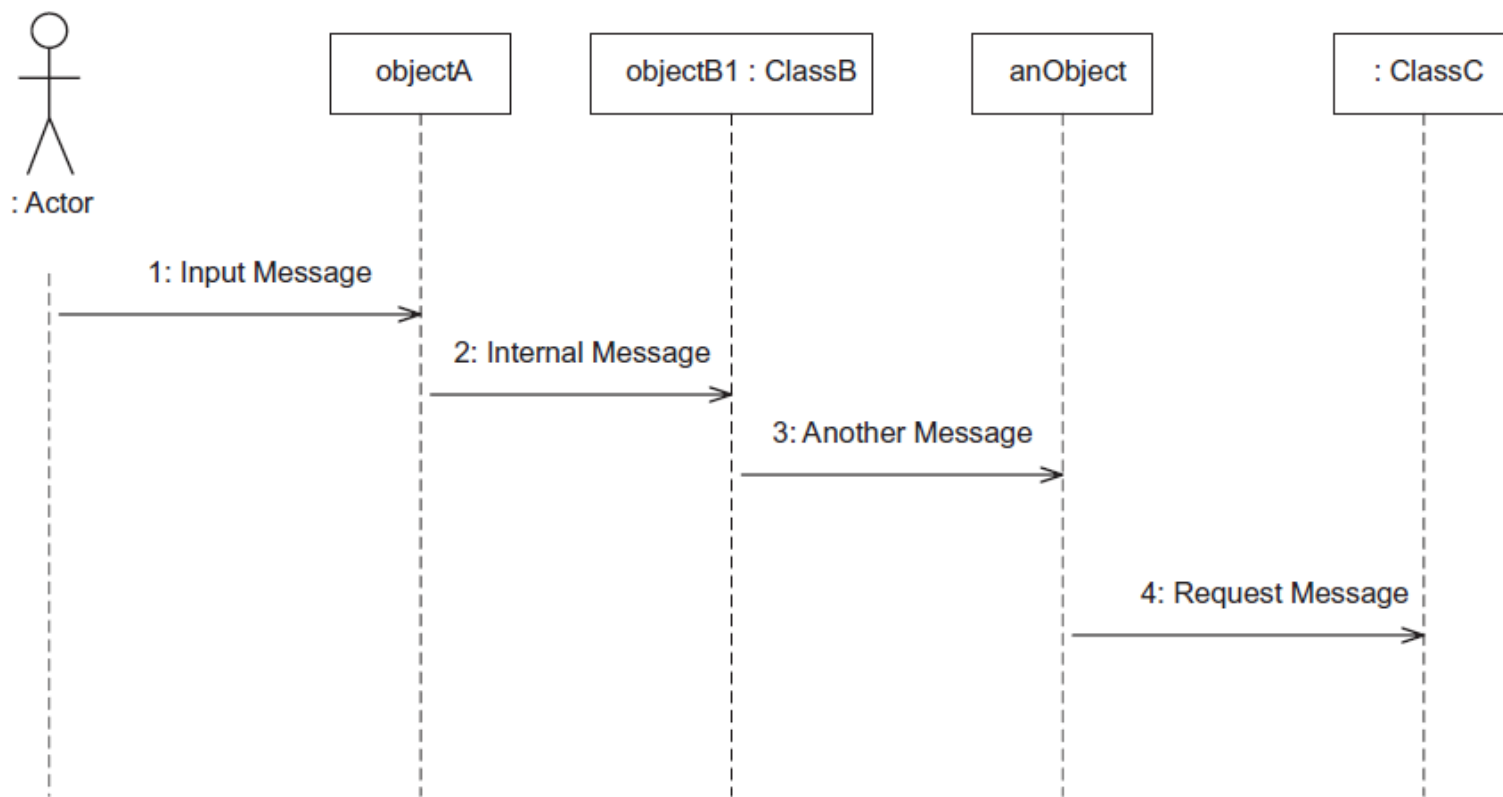
- 展示了合作对象间如何通过发送与接收消息进行动态的交互



UML表示法

○ 序列图

- 将对象交互通过时间序列的方式进行描述



UML表示法

● 状态机图（状态图）

○ 初始状态

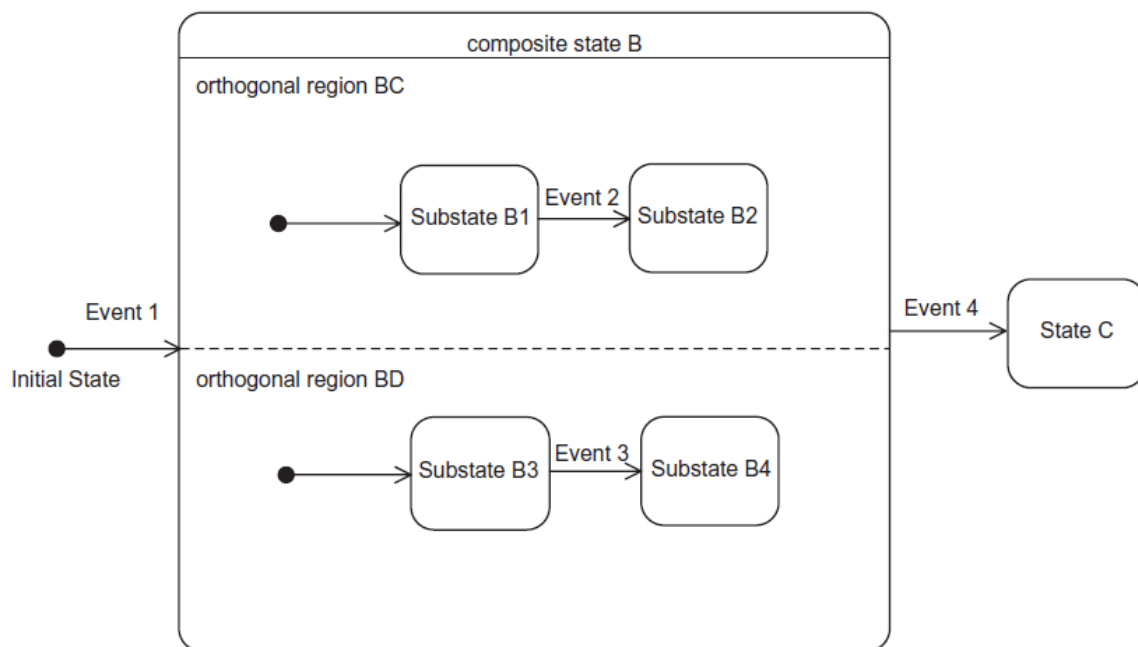
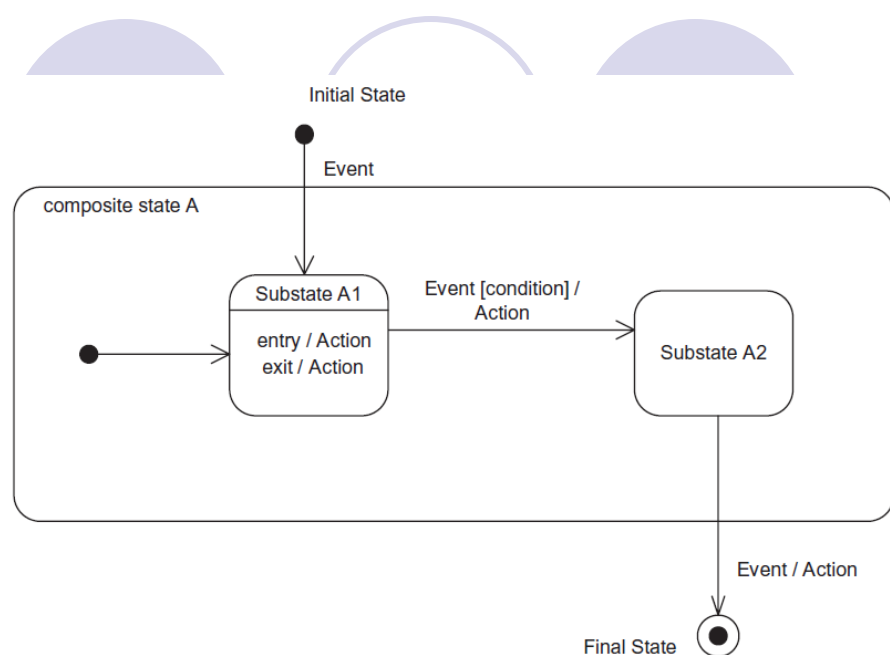
○ 终结状态

○ 弧（事件[条件]/动作）

○ 状态的动作

● 进入动作

● 退出动作



UML表示法

● 包图

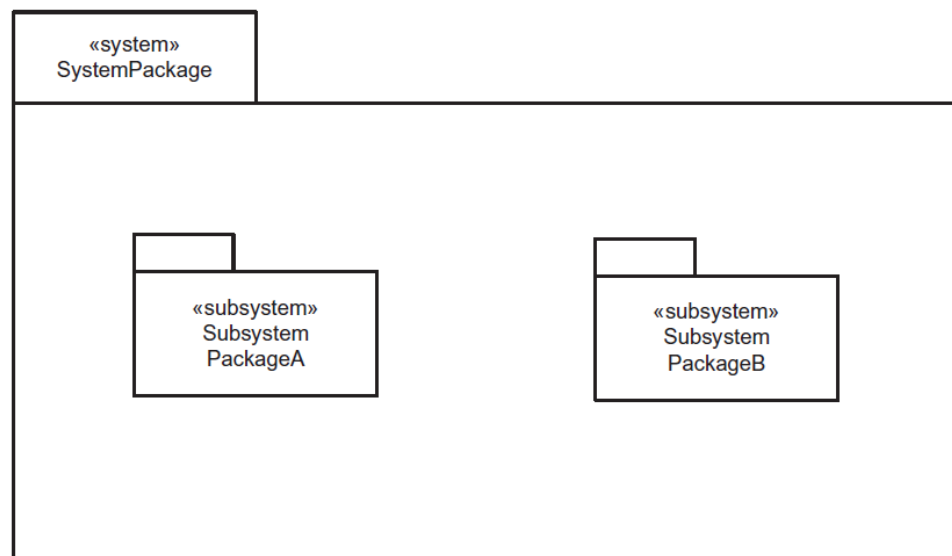
○ 包是一组建模元素的一个子系统

○ 包的关系

● 依赖

● 泛化/特化

○ 包可以容纳类、对象或用例



UML表示法

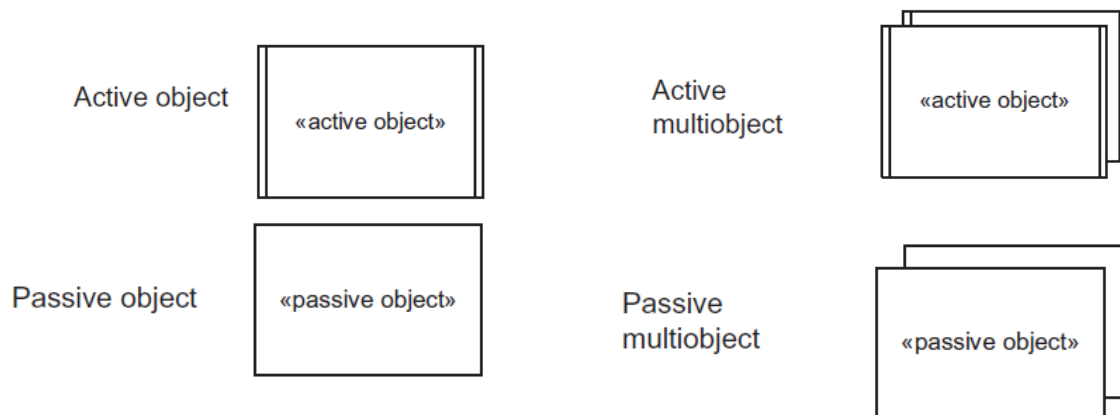
● 并发通信图

○ 主动对象

- 用于描绘一个并发对象、进程、线程或任务
- 拥有自己的控制线程，并能与其他对象并发执行

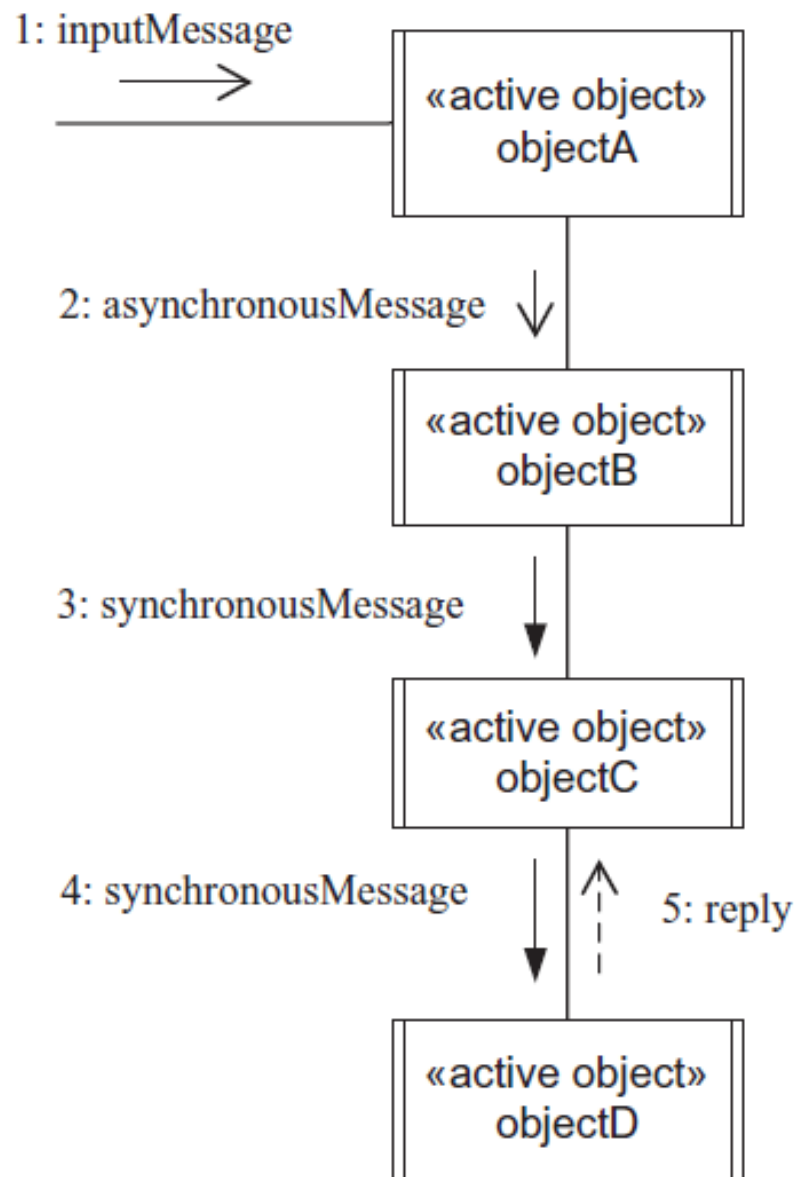
○ 被动对象

- 不具有控制线程，只有在其他对象调用其方法时才会执行



UML表示法

● 并发通信图



UML表示法

● 部署图

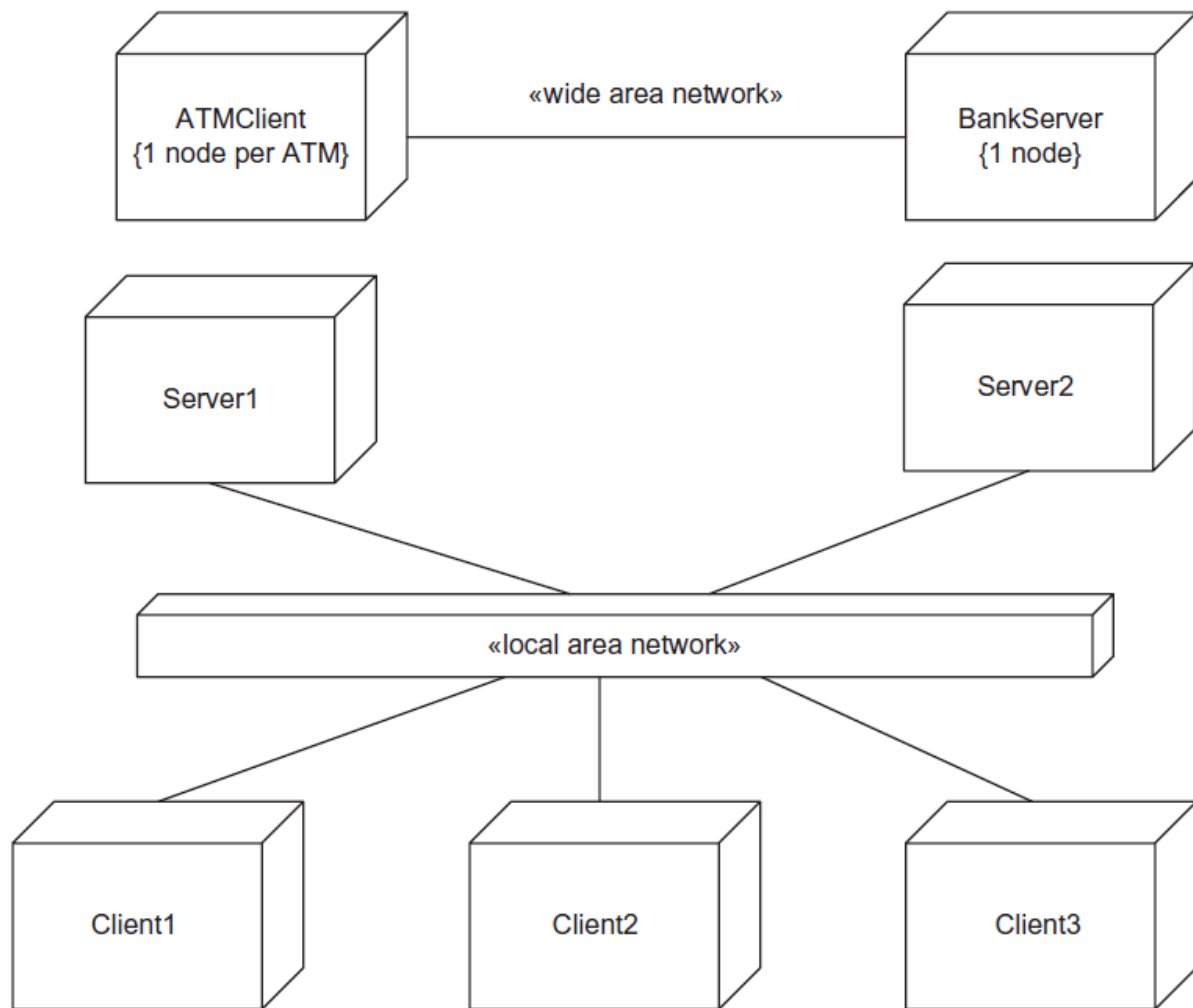
○ 以物理系统自

○ 本质

○ 结点

● 约束

● 连接



UML表示法

● UML扩展机制

○ 构造型

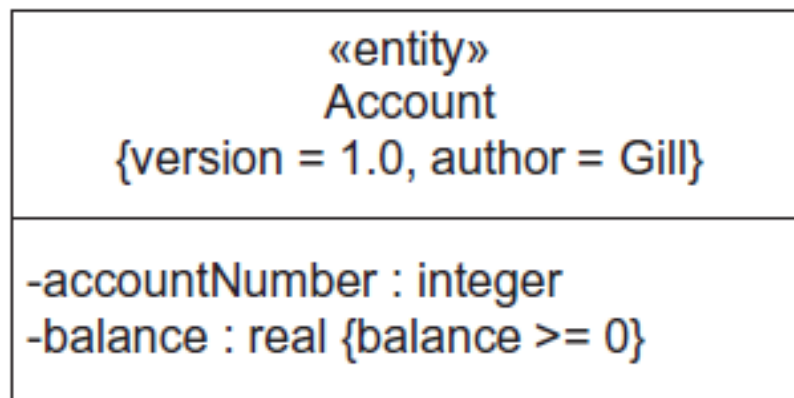
● <<stereotype>>

○ 标记值

● {标记=值}

○ 约束

● {expression}





西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

THE END